Step 3 — Report Progress as a Trace — Solution

> Back to the step · Hint 1 · Hint 2

Story: return a short revision **trace** alongside the summary, so a caller waiting on a slow, multi-round call can see what happened — without breaking the "one tool call" illusion. **If not:** the caller stares at a black box and then gets only the bare final string.

`src/formatting.py` (new)

Story: two pure formatters. `format_result` turns a `PipelineResult` into the "Deep research on / Revision trace / Status / Summary" block. `format_partial` (used in Step 4) turns a `Progress` holder into a marked partial answer. Pure functions, so they unit-test with no model. **If not:** formatting tangled inside the tool body couldn't be tested on its own.

```
# src/formatting.py
# Turn pipeline output into the string the MCP tool returns. Pure functions,
# so they're unit-testable with no model.
def format_result(result) -> str:
    lines = [f"Deep research on: {result.topic}", "", "Revision trace:"]
    for entry in result.rounds:
        state = "approved" if entry["approved"] else "rejected"
        lines.append(f"  Round {entry['round']}: {state}")
        for issue in entry["issues"]:
            lines.append(f"    - {issue}")
    if result.approved:
        status = f"approved after {len(result.rounds)} round(s)."
    else:
        status = (f"not approved; stopped at the "
                  f"{len(result.rounds)}-round cap.")
    lines.append("")
    lines.append(f"Status: {status}")
    lines.append("")
    lines.append("Summary:")
    lines.append(result.final_summary)
    return "\n".join(lines)


def format_partial(topic, progress) -> str:
    draft = progress.latest_draft or "(no draft produced yet)"
```

```
    return (
        f"Deep research on: {topic}\n\n"
        f"Status: PARTIAL - the pipeline timed out; returning the best draft "
        f"so far ({len(progress.rounds)} round(s) completed).\n\n"
        f"Draft so far:\n{draft}"
    )
```

`mcp_server.py` — return the formatted trace

Pass a `Progress` holder into `run_pipeline` (so Step 4 can read the best draft on timeout), and return `format_result(...)` instead of the bare summary:

```
# mcp_server.py  (Step 3 version)
from src.formatting import format_result
from src.pipeline import Progress, run_pipeline
# ...

@mcp.tool()
def deep_research(topic: str) -> str:
    """Research a topic through a fact-checked, multi-round pipeline; returns
    a verified summary plus a short revision trace."""
    logger.info(f"deep_research: {topic}")
    progress = Progress()
    result = run_pipeline(MODEL, topic, CONFIG.max_revision_rounds, progress)
    return format_result(result)
```

Now a `mcp dev mcp_server.py` call to `deep_research` returns the full block — the caller can *see* that round 1 was rejected (and why) and round 2 approved, without ever touching the pipeline's internals.

The test

```
# tests/unit/test_formatting.py
# The trace formatter, on a hand-made result — no model. Run from the repo
# root: python -m tests.unit.test_formatting
from src.formatting import format_partial, format_result


class FakeResult:
    def __init__(self, topic, final_summary, approved, rounds):
        self.topic = topic
        self.final_summary = final_summary
```

```
        self.approved = approved
        self.rounds = rounds


class FakeProgress:
    def __init__(self, latest_draft, rounds):
        self.latest_draft = latest_draft
        self.rounds = rounds


def test_result_shows_trace() -> None:
    result = FakeResult(
        "RAG", "RAG is from Meta AI, 2020.", True,
        [{"round": 1, "approved": False, "issues": ["wrong date"]},
         {"round": 2, "approved": True, "issues": []}],
    )
    out = format_result(result)
    ok = ("Round 1: rejected" in out and "- wrong date" in out
          and "Round 2: approved" in out
          and "approved after 2 round(s)" in out
          and "RAG is from Meta AI, 2020." in out)
    print(f"case 1 (trace + status + summary) -> "
          f"{'OK' if ok else 'WRONG'}")


def test_cap_status() -> None:
    result = FakeResult(
        "blue", "draft", False,
        [{"round": 1, "approved": False, "issues": ["x"]},
         {"round": 2, "approved": False, "issues": ["x"]}],
    )
    out = format_result(result)
    ok = "stopped at the 2-round cap" in out
    print(f"case 2 (cap status)              -> "
          f"{'OK' if ok else 'WRONG'}")


def test_partial() -> None:
    progress = FakeProgress("a half-finished draft", [{"round": 1}])
    out = format_partial("slow topic", progress)
    ok = "PARTIAL" in out and "a half-finished draft" in out
    print(f"case 3 (partial names the draft) -> "
          f"{'OK' if ok else 'WRONG'}")


if __name__ == "__main__":
    test_result_shows_trace()
```

```
    test_cap_status()
    test_partial()
```

Expected output:

```
case 1 (trace + status + summary) -> OK
case 2 (cap status)              -> OK
case 3 (partial names the draft) -> OK
```

The trace is what turns a slow black-box call into something a caller can trust: they see the fact-checker actually caught and fixed an error, not just a final string they have to take on faith.